Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Bu ders altı öğrenci sunumundan oluşur. Sunumlar dersin mikroservis kitabının (slaytlarda: *Microservices Patterns With examples in Java*) sonraki bölümlerini izler: CQRS ile sorgu gerçekleştirme, API Gateway deseni ve gerçekleştirilmesi, mikroservislerde test stratejileri, bir servis için birim testi yazma ve entegrasyon testi yazma. Sunan kişiler aşağıda "bir öğrenci" diye anılır. CQRS'in tasarımıyla ilgili planlanan ikinci sunum bu derste yapılmadı. Hoca bu derste sunumları yönetti, akademik bir ek açıklama yapmadı.

Genel Konular

- Birinci sunum: CQRS (Command Query Responsibility Segregation) deseni
  - Motivasyon: mikroservis mimarisinde bir sorgunun ihtiyaç duyduğu veri birden çok servise dağılmıştır. Varsayılan çözüm API composition'dır: sorguyu yapan taraf her servisi çağırır ve sonuçları bellekte birleştirir (in-memory join). Karmaşık sorgularda bu yaklaşım pahalı ve verimsiz olur; geliştirme süreci ürün yerine sorguları çözmeye kayar.
  - `findOrderHistory()` örneği: bir müşterinin sipariş geçmişi sipariş tarihine göre sıralı listelenmek istenir. Dönen her kayıtta sipariş numarası, sipariş durumu, toplam tutar ve teslimat zamanı bulunur. Bu bilgi için Accounting (ödeme), Delivery (teslimat) ve Order (sipariş) servislerinden gelen veriyi birleştirmek, sıralamak ve filtrelemek gerekir. Order ve Kitchen servisleri müşteri kimliği ve menü öğesi adı gibi alanlarla sorgulanabilirken Delivery ve Accounting servisleri yalnız müşteri kimliğini (consumer ID) destekler; bu da join'i zorlaştırır.
    - In-memory join: bütün servislerin cevapları çekilip bellekte birleştirilir. Büyük veri kümelerinde çok bellek ister ve verimsizdir.
    - ID ile sorgulama: bir servisten kimlikler alınıp diğer servisler bu kimliklerle sorgulanır. Bunun verimli olması için diğer servislerin toplu veri alma (bulk fetch) API'si sunması gerekir; yoksa tek tek istekler ağ trafiğini artırır ve performansı düşürür.
  - `findAvailableRestaurants()` örneği: belli bir adrese belli bir zamanda teslimat yapabilecek restoranları bulma. Temelinde coğrafi konum araması vardır. Bazı veritabanları bunu (iki nokta arası mesafe, en yakın noktalar, coğrafi indeks) kolayca destekler, bazıları desteklemez.
  - Öğrenciye göre sorgu tarafı ile komut tarafı aynı veritabanında çalıştığında, bir komut işlenirken aynı veriye başka bir sorgu gelebilir ve bu asenkron işleyiş veri tutarsızlığına yol açabilir. Bu yüzden sorgu ve komut taraflarının ayrılması gerekir.
  - CQRS tanımı (ekrandan, 00:10:00): kalıcı bir veri modelini ve onu kullanan modülleri iki parçaya ayırır.
    - Command side: create, update ve delete (CUD) işlemlerini uygular; veride değişiklik olduğunda domain event'leri yayınlar.
    - Query side: sorgular (örneğin HTTP GET istekleri) için ayrı bir veri modeli sağlar; verisini komut tarafının yayınladığı event'lere abone olarak senkronize eder.
    - CQRS olmayan yapıda tek servis, domain model (aggregate'lar) ve "query bypass" aynı veritabanını kullanır. CQRS yapısında command/domain model "command-side database"e yazar, aggregate'lar event yayınlar, query model içindeki event handler bu event'leri yakalayıp ayrı "query database"i günceller (ekrandan, 00:10:00).
  - Sorgu servisi (query service) (ekrandan, 00:11:30): CQRS yalnız bir servisin içinde

değil, ayrı sorgu servisleri tanımlamak için de kullanılır. Query service yalnız sorgu işlemlerinden oluşan bir API'ye sahiptir, komut işlemi içermez. Order History Service, `findOrderHistory()` ve `findOrder()` sorgularını sunar; Order, Kitchen, Delivery ve Accounting servislerinin yayınladığı Order/Ticket/Delivery/Accounting event'lerine abone olur ve "Order history view database"i günceller.
  - Avantajlar:
    * Mikroservis mimarisinde sorguların verimli gerçekleştirilmesi: API composition'ın pahalı in-memory join'i yerine önceden güncellenmiş görünüm sorgulanır.
    * Farklı sorguların verimli desteklenmesi: tek bir veritabanının sorgu yetenekleri sınırlı olabilir; CQRS birden çok görünüm tanımlayarak her birini belli sorgular için uygun biçimde tasarlar.
    * Ayrı sorumluluklar (separation of concerns): komut ve sorgu tarafları için ayrı kod modelleri ve veritabanı şemaları tanımlanır; iki taraf da daha basit ve bakımı kolay olur, veritabanı yükü bölünür.
  - Dezavantajlar:
    * Daha karmaşık mimari: geliştiriciler görünümleri güncelleyen ve sorgulayan sorgu tarafı servislerini yazmak zorundadır; ek veri depolarının yönetimi operasyonel karmaşıklık getirir; farklı türde veritabanları kullanılıyorsa geliştirme ve operasyon ekiplerinin işi zorlaşır.
    * Replikasyon gecikmesi (replication lag): komut tarafı event yayınladıktan sonra sorgu tarafındaki görünüm hemen güncellenmez. İstemci bir nesneyi güncelleyip hemen sorgularsa eski sürümü görebilir. Çözüm olarak komut ve sorgu API'leri istemciye sürüm bilgisi verebilir; istemci böylece sorgu tarafının güncel olmadığını anlar.
- İkinci sunum: API Gateway deseni
  - Problem: istemcinin servislerle doğrudan iletişimi. Küçük uygulamalarda uygun olabilir; büyük sistemlerde istemcinin çok sayıda istek yapmasına, farklı protokolleri yönetmesine ve kodunun karmaşıklaşmasına yol açar. İstemci ile mikroservisler arasında sıkı bağ oluşur, bakım zorlaşır. Kimlik doğrulama, yetkilendirme, izleme gibi işlerin her mikroserviste ayrı ayrı yapılması verimsizdir.
  - Tanım: istemcilerin isteklerini mikroservislere yönlendiren; kimlik doğrulama, yük dengeleme, izleme gibi görevleri üstlenen ve istemcilere tek bir giriş noktası (single entry point) sunan bileşen.
  - Gateway routing (ekrandan, 00:20:00): tek bir uç nokta üzerinden birden fazla mikroservise (şekilde Customer, ShoppingCart, Discount, Order) istek yönlendirme. Mikroservis değişiklikleri istemci kodunu etkilemez, yalnız yönlendirme yapılandırması değişir; gateway arkadaki mikroservisleri istemciden soyutlar. Öğrenciye göre kitapta API sürüm yönetimini de kolaylaştırdığı yazar.
  - API composition: gateway birden fazla servisi çağırıp sonuçları tek bir yanıtta birleştirir; örneğin sipariş ayrıntıları için istemci birden çok istek yapmak yerine tek istek yapar.
  - Protocol translation: örneğin istemciden gelen HTTP isteği arka uçta gRPC'ye çevrilir.
  - İstemciye özel API: üçüncü taraf uygulama tam sipariş ayrıntısını isterken mobil istemci yalnız bir alt kümeyi isteyebilir; daha gelişmiş yaklaşım her istemciye ayrı API sunmaktır.
  - Edge function'lar: kimlik doğrulama (authentication), yetkilendirme (authorization), hız sınırlama (rate limiting), önbellekleme (caching), metrik toplama, istek loglama.
  - Mimari (ekrandan, 00:23:40): API katmanı bir ya da daha çok bağımsız API modülünden oluşur (Mobile API, Browser API, Public API), her modül belli bir istemci için API uygular. Ortak katman (common layer) tüm API'ler için ortak işlevselliği, örneğin kimlik doğrulama gibi edge function'ları içerir. API modülü istekleri ya doğrudan bir servise yönlendirir ya da birden çok servisi çağırıp sonuçları birleştirir.
  - Sahiplik modeli (ownership model): her istemci ekibi kendi API modülünden sorumludur; API Gateway ekibi yalnız ortak modülden ve gateway'in operasyonundan sorumludur. Merkezi tek bir gateway ekibi geliştirme hızını engelleyebilir; bu modelde ekipler API'lerini bağımsız değiştirir.
  - Backends for Frontends (BFF): her istemci türü için ayrı API gateway. Sorumluluklar netleşir, ölçeklenebilirlik artar; ortak işlevsellik paylaşılan bir kütüphaneyle sağlanır, bütün gateway'ler aynı teknoloji yığınını kullanır. Netflix örneği: başlangıçta tek API

kullanılmış, farklı cihaz türlerinin ihtiyaçları karşılanamayınca cihaz türlerine özel yaklaşıma geçilmiş. Slayta göre Netflix'in açık kaynaklı gateway'i Zuul dinamik yönlendirme, izleme ve güvenlikten sorumludur (ekrandan, 00:28:00).
  - Tasarım konuları: performans ve ölçeklenebilirlik (asenkron, bloke olmayan G/Ç ölçeklenir ama karmaşıktır), reaktif programlama ile bakımı yapılabilir kod, kısmi hataların ele alınması, mimaride "iyi bir vatandaş" olmak (servis keşfi, gözlemlenebilirlik desenleri).
  - Avantajlar (ekrandan, 00:30:00): basitleştirilmiş API, esneklik, güvenlik, performans ve ölçeklenebilirlik (caching, rate limiting), protokol çevirisi.
  - Dezavantajlar: geliştirme ve yönetim yükü, tek hata noktası riski, ek gecikme, yüksek trafik ve karmaşıklık. Önerilen hafifletme BFF desenidir.
- Üçüncü sunum: API Gateway'in gerçekleştirilmesi
  - Monolitik mimaride FTGO uygulamasında sipariş ayrıntısı tek bir `getOrderDetails` çağrısıyla alınır. Mikroservis mimarisinde aynı bilgi için örneğin dört ayrı servis çağrılır; istemcinin her servisin URL'ini bilmesi gerekir. API Gateway tek giriş noktası olarak bu sorunu çözer: yönlendirme, kimlik doğrulama, yük dengeleme, veri birleştirme (aggregation), rate limiting, izleme, loglama ve protokol çevirisi.
  - Hazır ürünler (ekrandan, 00:40:00):
    * AWS API Gateway: temel ihtiyaçlar için sağlam ama API composition gibi gelişmiş özellikleri yok. Lambda, DynamoDB, CloudWatch gibi diğer AWS servisleriyle entegre çalışır.
    * AWS Application Load Balancer: basit yük dengeleme, sınırlı gateway özelliği; edge function ve API composition yok.
    * Kong/Traefik gibi API gateway ürünleri: açık kaynak, özelleştirilebilir ve esnek ama kurulum, yapılandırma ve bakım için operasyonel çaba ister.
  - Kendi gateway'ini geliştirmek: iki kritik nokta vardır, yönlendirme kurallarını doğru tanımlamak ve HTTP proxy davranışını (header'lar dahil) doğru uygulamak. Öğrenciye göre kitap bir gateway çatısı olarak Spring Cloud Gateway'i öne çıkarır: yönlendirme, edge function ve veri birleştirme sade biçimde yazılır; reaktif ve asenkron yapısı ölçeklenebilirlik sağlar.
  - Dört temel görev: request routing, API composition, edge functions (en temeli authentication) ve protocol translation. Slaytlarda Python (FastAPI tarzı) örnek kodlar gösterildi (ekrandan, 00:44:25 ve 00:45:20); bkz. Detaylı Açıklamalar.
- Dördüncü sunum: mikroservis mimarisinde test stratejileri
  - Test: yazılımın belli gereksinimlere ve beklentilere uygun çalıştığını doğrulama işlemi. Mikroservislerde servisler arası iletişimi doğrulamak ayrıca önemlidir.
  - Önemi: hataların erken tespiti, güvenilir sistem, hızlı ve sürekli değişimde kodun sistemi bozmadığını doğrulama, müşteri memnuniyeti, maliyet tasarrufu.
  - Otomatik testler test çerçeveleriyle yazılır: Java'da JUnit, Python'da PyTest, C#'ta NUnit (ekrandan, 00:50:00). Faydaları: kolay kullanım, otomasyon, hız ve güvenilirlik.
  - Test yapısı (ekrandan, 00:50:00): Test runner testleri çalıştırır; test class bir ya da daha çok test metodu içerir, her metot bir senaryodur. Fixture dört aşamadır: setup (ön koşullar), execute (test edilen kod, SUT = System Under Test, çalıştırılır), verify (sonuç doğrulanır), teardown (temizlik).
  - Test ikizleri (test doubles) (ekrandan, 00:52:35): bağımlılıkları taklit eden nesneler. Stub, SUT'un bağımlılıktan beklediği sabit değeri döndürür. Mock, bağımlılığın doğru biçimde çağrılıp çağrılmadığını doğrular.
  - Öğrenciye göre kitapta Brian Marick'in test sınıflandırması anlatılır: testlerin neden yapıldığı, hangi hedefe hizmet ettiği ve hangi türün hangi amaca uygun olduğu.
  - Test piramidi (ekrandan, 00:55:10): tabanda birim (unit) testleri (iş mantığını test eder; hızlı, güvenilir, ucuz), üstünde entegrasyon testleri (servisin bağımlılıklarıyla iletişimini doğrular), bileşen (component) testleri (bir servisin kabul testleri) ve tepede uçtan uca (end-to-end) testler (uygulamanın kabul testleri; yavaş, kırılgan, pahalı). Yukarı çıktıkça test sayısı azalmalıdır.
  - Deployment pipeline: kodun geliştiricinin masasından üretime kadar otomatik süreci. Aşamalar: commit öncesi birim testleri, commit testleri, entegrasyon testleri, bileşen testleri, üretime dağıtım (CI/CD).
  - Test türleri: birim, entegrasyon, bileşen, uçtan uca, sözleşme (contract, tüketici ile sağlayıcı arasındaki API sözleşmesi), dayanıklılık (resilience), performans, güvenlik

testleri. Mikroservislerin bağımsız yapısı testlerin paralel çalıştırılmasını mümkün kılar.

- Beşinci sunum: bir servis için birim testi yazma
  - Birim testi bir sınıfı, kontrolcüyü, servisi ya da entity'yi bağımsız ve hızlı biçimde test eder. Faydaları: hataların erken tespiti, kod kalitesi, güvenli refactoring, uzun vadede düşük maliyet.
  - Solitary unit test (ekrandan, 01:10:00): birimin bağımlılıkları mock, stub ya da fake ile değiştirilir; test yalnız birimin davranışına odaklanır.
  - Sociable unit test: birim gerçek bağımlılıklarıyla birlikte test edilir.
  - Kitaptaki şekle göre (ekrandan, 01:12:20) controller ve service sınıfları genelde solitary, entity ve value object gibi domain nesneleri (şekilde saga da) sociable birim testleriyle test edilir.
  - AAA deseni: Arrange (bağımlılıkları ve nesneyi hazırla, gerekirse mock'la), Act (test edilen metodu çağır), Assert (sonucu ya da beklenen istisnayı doğrula).
  - Örnek: `Calculator.add(3, 5)` için `assertEquals(8, result)` (ekrandan, 01:15:10). Servis örneği: `UserService.getUserName` için `UserRepository` mock'lanır; kullanıcı yoksa "User not found", varsa kullanıcının adı döner; iki senaryo için iki test yazılır (ekrandan, 01:16:50).
- Altıncı sunum: entegrasyon testi yazma
  - Entegrasyon testi piramitte birim testinin hemen üstünde, uçtan uca testin altındadır. Servisin altyapı servisleri ve diğer uygulama servisleriyle doğru iletişim kurduğunu ve sözleşmelere (contract) uyduğunu doğrular. Beklenmeyen davranışı uçtan uca testlerden önce yakalamak maliyet ve zaman kazandırır; ekip içi işbirliğini kolaylaştırır, regresyonu önler.
  - Kitaptaki strateji (ekrandan, 01:20:00): servisin tamamı değil, iletişimi gerçekleştiren adaptör sınıfları test edilir. Order Service için dört etkileşim:
    - 1: API Gateway (consumer, `OrderServiceProxy`) ile Order Service (provider, `OrderController`) arasında REST tabanlı request/response.
    - 2: Order Service (provider, `OrderDomainEventPublisher`) event yayınlar, Order History Service (consumer, event handler'lar) tüketir: publish/subscribe.
    - 3: `OrderRepository` ile veritabanı: kalıcılık (persistence).
    - 4: Order Service (consumer, `KitchenServiceProxy`) ile Kitchen Service (provider, command handler) arasında komut ve yanıt kanallarıyla asenkron request/response.
  - Consumer tarafı, provider'ı taklit eden mock sunucularla; provider tarafı, adaptörün bağımlılıklarını taklit eden mock'larla test edilir. Karşı tarafı fiziksel olarak ayağa kaldırmak gerekmez, testler hızlanır.
  - Persistence entegrasyon testi: setup (şema oluşturulur, veritabanı bilinen bir duruma getirilir), execute (veritabanı işlemi), verify (veritabanından okunan nesne beklenen durumla karşılaştırılır), teardown (değişiklikler geri alınır; kitapta isteğe bağlı). Örnek Listing 10.1 (ekrandan, 01:24:15).
  - Spring Cloud Contract: sözleşme tabanlı test aracı. Consumer tarafı için WireMock ile provider'ı taklit eden sunucu yapılandırır; provider tarafı için test kodu üretir (REST Assured MockMvc ile controller'ı test eder). REST sözleşmesi Listing 10.2, provider taban sınıfı Listing 10.3, consumer testi Listing 10.4, publish/subscribe sözleşmesi Listing 10.5 (ekrandan).
  - Publish/subscribe testi: provider tarafında `OrderDomainEventPublisher`'ın sözleşmeye uygun event yayınladığı; consumer tarafında Order History event handler'ının sözleşmedeki örnek event'leri doğru tükettiği doğrulanır.
  - Asenkron request/response testi: requester (komut gönderen) ve replier (komutu işleyip yanıt veren) komut kanalının adı ile komut ve yanıt mesajlarının yapısı üzerinde anlaşır. Provider tarafında Kitchen Service command handler'ının komutu işleyip doğru yanıt verdiği, consumer tarafında `KitchenServiceProxy`'nin sözleşmeye uygun komut gönderdiği ve örnek yanıtları doğru işlediği doğrulanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu derste hoca akademik bir vurgu yapmadı; sunumları sırayla yönetti. Sunumlarda öne çıkan noktalar aşağıdadır.
- CQRS'in amacı sorguyu komuttan ayırmaktır

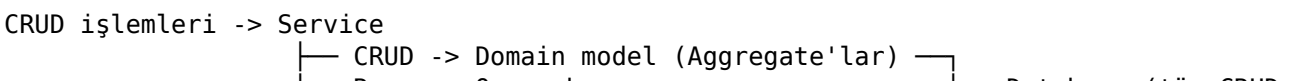
  - Komut tarafı yazar ve event yayınlar, sorgu tarafı event'lere abone olarak kendi görünümünü günceller. Bedeli karmaşıklık ve replikasyon gecikmesidir.
- API Gateway tek giriş noktasıdır ama tek hata noktası da olabilir
  - Çözüm önerisi BFF: her istemci türüne ayrı gateway.
- Test piramidi
  - Çok sayıda birim testi, daha az entegrasyon/bileşen testi, en az uçtan uca test.
- Entegrasyon testinde servisin tamamı değil adaptör sınıfları test edilir
  - Sözleşmeler (contract) consumer ve provider testlerinin ortak kaynağıdır; Spring Cloud Contract iki tarafın testini de bu sözleşmelerden üretir.

Kısa Tekrar Notları

- CQRS = Command Query Responsibility Segregation. Command side: CUD + domain event yayını. Query side: ayrı model ve ayrı veritabanı, event handler ile güncellenir.
- API composition sorunu: büyük veri kümelerinin bellekte birleştirilmesi pahalıdır; ID ile sorgu için bulk fetch API gerekir.
- Query service: yalnız sorgu API'si (örnek Order History Service, `findOrderHistory()`, `findOrder()`).
- CQRS artıları: verimli sorgu, farklı sorgu türleri için uygun görünümler, ayrı sorumluluklar. Eksileri: karmaşıklık, replikasyon gecikmesi (çözüm: sürüm bilgisi).
- API Gateway görevleri: request routing, API composition, protocol translation, istemciye özel API, edge function'lar (authN, authZ, rate limiting, caching, metrik, loglama).
- API Gateway mimarisi: API katmanı (istemciye özel modüller) + ortak katman.
- Sahiplik: istemci ekipleri kendi API modülü; gateway ekibi ortak modül ve operasyon.
- BFF: istemci türü başına bir gateway (Netflix, Zuul).
- Hazır çözümler: AWS API Gateway (composition yok), AWS ALB (çok sınırlı), Kong/Traefik (esnek, operasyon yükü). Kendi gateway'in: Spring Cloud Gateway.
- Fixture: setup → execute → verify → teardown.
- Stub sabit değer döndürür; mock çağrının doğru yapıldığını doğrular.
- Test piramidi: unit → integration → component → end-to-end.
- Solitary: bağımlılıklar mock'lanır (controller, service). Sociable: gerçek bağımlılıklarla (entity, value object).
- AAA: Arrange, Act, Assert.
- Entegrasyon testinin dört etkileşimi: REST request/response, publish/subscribe, persistence, asenkron request/response.
- Spring Cloud Contract: consumer için WireMock stub sunucu, provider için üretilen test (REST Assured MockMvc).

Detaylı Açıklamalar

- CQRS neden gerekir?
  - Mikroservis mimarisinde her servis kendi verisine sahiptir. `findOrderHistory()` gibi bir sorgu Order, Kitchen, Delivery ve Accounting servislerinin verisine birlikte ihtiyaç duyar. API composition ile bu servislerden gelen cevaplar bellekte birleştirilir, sıralanır ve filtrelenir. Sonuç kümesi büyükse bu yöntem çok bellek ister. Kimlikle sorgulama yöntemi ise diğer servislerde bulk fetch API'si yoksa çok sayıda küçük istek üretir. CQRS bu sorunu, sorgunun ihtiyaç duyduğu veriyi event'lerle önceden güncellenen ayrı bir görünüm veritabanında tutarak çözer. Sorgu artık tek bir veritabanına gider.
  - Coğrafi sorgu örneğinde (`findAvailableRestaurants()`) sorun, servisin kendi veritabanının bu sorguyu verimli desteklemeyebilmesidir. CQRS ile sorgu tarafı için coğrafi indeksleri destekleyen ayrı bir veritabanı seçilebilir.
- CQRS'in yapısı (ekrandan, 00:10:00)
  - CQRS olmayan yapı:



- CQRS yapısı:

```
CRUD işlemleri -> Service
                  ├── CUD -> Command/domain model (Aggregate'lar) -> Command-side dat
                  │                  | Events
                  │                  v
                  └── R   -> Query model (Event handler) -> Query database
```

- Komut tarafı her yazmadan sonra event yayınlar; sorgu tarafının event handler'ı bu event'leri yakalayıp sorgu veritabanını günceller.

- API Gateway gerçekleştirme örnekleri (ekrandan, 00:44:25 ve 00:45:20)
  - Öğrencinin "request routing" başlığıyla gösterdiği kod:

```python
@app.get("/orders/{order_id}")
async def get_order(order_id: int, user: dict = Depends(authenticate_token)):
    async with httpx.AsyncClient() as client:
        order_resp = await client.get(f"{BACKEND_SERVICES['order']}/orders/{order_id}"
        consumer_resp = await client.get(f"{BACKEND_SERVICES['consumer']}/consumers/{o
        delivery_resp = await client.get(f"{BACKEND_SERVICES['delivery']}/deliveries/-
    return {
        "order": order_resp.json(),
        "consumer": consumer_resp.json(),
        "delivery": delivery_resp.json()
    }
```

  - Ders sonrası not: Öğrenci bu kodu bir request routing örneği olarak anlattı. Kod üç servisi çağırıp yanıtları tek bir JSON'da birleştirdiği için büyük olasılıkla daha çok API composition örneğidir; saf yönlendirmede istek tek bir uygun servise iletilir. Kod ayrıca `Depends(authenticate_token)` ile edge function'ı (kimlik doğrulamayı) devreye sokar.
  - Edge function (kimlik doğrulama) örneği:

```python
def authenticate_token(request: Request):
    token = request.headers.get("Authorization")
    if not token:
        raise HTTPException(status_code=401, detail="Missing Authorization Header")
    try:
        scheme, _, credentials = token.partition(" ")
        if scheme.lower() != "bearer" or not credentials:
            raise HTTPException(status_code=401, detail="Invalid Authorization Header
        payload = jwt.decode(credentials, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid Token")
```

  - Fonksiyon `Authorization` başlığındaki "Bearer " değerini ayırır, JWT'yi çözer; başlık yoksa, biçim yanlışsa ya da token geçersizse 401 döner. Böylece yetkisiz istekler arka uç servislerine hiç ulaşmaz.

- Birim testi örneği (ekrandan, 01:00:00)

```java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

public class CalculatorTest {
    @Test
    public void testAdd() {
        Calculator calculator = new Calculator();
        int result = calculator.add(5, 10);
```

```
        assertEquals(15, result);
    }

    @Test
    public void testAddNegativeNumbers() {
        Calculator calculator = new Calculator();
        int result = calculator.add(-5, -10);
        assertEquals(-15, result);
    }
}
```

- Bileşen testi ve mock (ekrandan, 01:03:20)
  - `PaymentService`, `PaymentGateway`'e bağımlıdır. Testte `@Mock PaymentGateway` ve `@InjectMocks PaymentService` kullanılır; `when(paymentGateway.process(any(Payment.class))).then` ile gateway taklit edilir, `processPayment(new Payment(100, "credit_card"))` çağrısının `true` döndüğü `assertTrue` ile doğrulanır. Gerçek ödeme gateway'i çalıştırılmaz.
- Solitary birim testi örneği (ekrandan, 01:16:50)

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public String getUserName(int userId) {
        User user = userRepository.findUserById(userId);
        return (user != null) ? user.getName() : "User not found";
    }
}
```

  - Testte `@Mock UserRepository` kullanılır. Birinci test: `given(userRepository.findUserById(123)).w` `getUserName(123)` sonucunun "User not found" olduğu doğrulanır. İkinci test: repository id'si 123 olan bir `User` döndürür; `getUserName(123)` sonucunun bu kullanıcının adı olduğu doğrulanır. Testler given/when/then yorumlarıyla AAA yapısını izler.
- Entegrasyon testi ve Spring Cloud Contract
  - REST sözleşmesi (ekrandan, 01:27:30, Listing 10.2): `GET /orders/1223232` isteğine `200` durum kodu, `Content-Type: application/json;charset=UTF-8` başlığı ve `{"orderId" : "1223232", "state" : "APPROVAL_PENDING"}` gövdesiyle yanıt verilir.
  - Consumer tarafı testi (ekrandan, 01:30:00, Listing 10.4): `@AutoConfigureStubRunner` annotasyonu Spring Cloud Contract'a WireMock sunucusunu rastgele bir portta çalıştırmasını ve Order Service sözleşmeleriyle yapılandırmasını söyler. `shouldVerifyExistingCustomer` testi `findOrderById("1223232")` sonucunun kimliğini ve `APPROVAL_PENDING` durumunu doğrular; `shouldFailToFindMissingOrder` testi `findOrderById("555")` için `OrderNotFoundException` bekler.
  - Provider tarafı (ekrandan, 01:28:40, Listing 10.3): üretilen testlerin taban sınıfı `HttpBase`, `OrderService` ve `OrderRepository`'yi mock'lar, `OrderController`'ı bunlarla kurar, `findById(1223232L)` çağrısına örnek siparişi döndürür ve `RestAssuredMockMvc.standaloneSetup(...)` ile Spring MVC'yi yapılandırır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.